

Организация обработки задач в проекте AIS

Дата: 15.09.2026

1. Общая схема обработки задачи

Обработка любой задачи строится вокруг **Jira-задачи** (SYBASE- / SUPRT-), которая является точкой входа. Вокруг неё организуется рабочая папка с фиксированной структурой и цикл перемещения объектов по папкам по мере их готовности.

Жизненный цикл задачи

1. **Поступление задачи** — в Jira создана задача с техническим заданием (ТЗ).

2. **Создание рабочей папки** — по имени задачи в корне задач:

- C:\AIS\1 Release\<задача> — для задач SYBASE- / SUPRT-;
- C:\AIS\AI\Prod\tasks\<задача> — для остальных задач.

3. **Выгрузка оригиналов** в папку Git_YYYY_MM_DD :

- SQL-объекты выгружаются из dev_golden через SQL_exp_single.bat ;
- PB-объекты копируются из PB_Current\<lib>\ или C:\SRC125\gold\<lib>\ (только чтение).

4. **Разработка и размещение** изменённых объектов в Test_YYYY_MM_DD — сюда попадают только ГОТОВЫЕ объекты (без диагностических вставок).

5. **Диалоговое сравнение** в `Dial_YYYY_MM_DD` — пары «старый/новый» код для проверки side-by-side.
6. **Диагностика** (при необходимости) в `DIAG_YYYY_MM_DD_hh_mm` — отдельная папка с инструкцией по тестированию в формате `.md`.
7. **Готовые объекты** в `Ready_YYYY_MM_DD` — только по команде после тестирования.
8. **Ввод в продакшн** — папка `Read_YYYY_MM_DD`.
9. **Фиксация изменений** — `Describe\<объект>.txt` и `ПЛАН_РЕАЛИЗАЦИИ.txt`.

Структура рабочей папки

<задача>\

- |— `Git_YYYY_MM_DD\` — оригиналы (выгрузка из продакшна)
- |— `Test_YYYY_MM_DD\` — готовые изменённые объекты
- |— `Test_YYYY_MM_DD__hh_mm\` — повторные сборки за ту же дату
- |— `Dial_YYYY_MM_DD\` — старый vs новый код
- |— `DIAG_YYYY_MM_DD_hh_mm\` — диагностические объекты + ИНСТРУКЦИЯ.md
- |— `Ready_YYYY_MM_DD\` — после тестирования (по команде)
- |— `Read_YYYY_MM_DD\` — готово к продакшну
- |— `Describe\` — описания изменений + анализ
 - | |— `<объект>.txt`
 - | |— `ПЛАН_РЕАЛИЗАЦИИ.txt`
- |— `rules/pending-tasks.md` — якорь задачи (статус для всех сессий)

> Папки `Test_ / Dial_` раскладываются по подпапкам с именем библиотеки, например `golden_bill\`, `golden_sprw\`, а для SQL — `golden\Functions\`, `golden\Procedure\`.

2. Что используется вместо SDD

Классический документ проектирования (SDD) в проекте заменён набором лёгких артефактов, которые ведёт каждая задача:

2.1. План реализации (ПЛАН_РЕАЛИЗАЦИИ.txt)

Основной документ задачи. Ведётся в папке `Describe\`. Содержит:

- **Цель** задачи и статус (в работе / отложено / завершено);
- **Этапы** — нумерованный список с чек-боксами:
 - `[ ]` — не начат;
 - `[~]` — в работе;
 - `[x]` — готов;
 - `[!]` — отложен на дорожную LLM;
- у каждого этапа — результат и отметка, какой LLM выполнен;
- **СЛЕДУЮЩИЙ ШАГ** — с чего продолжить при возобновлении;
- **ОЧЕРЕДЬ ДОРОГИХ ПОДЗАДАЧ** — список `[!]`-этапов с обоснованием.

2.2. Описания объектов (Describe\<объект>.txt)

На каждый изменяемый объект заводится файл с описанием:

Задача: SYBASE-NNNN

Дата: ДД.ММ.ГГГГ ЧЧ:ММ

Объект: <имя_объекта>

=== Анализ ===

=== Рекомендации ===

=== Изменения ===

2.3. База знаний MCP (ais-catalog)

Сводные записи сохраняются через `ais-catalog_save_knowledge`. Одна запись на задачу/объект со структурой: анализ → вывод → карта воздействия → точка возобновления. Теги используются для точного поиска.

2.4. TaskPlan (для объёмных задач)

TaskPlan — сервис управления объёмными задачами, заменяющий классическую проектную документацию:

- **Show-TaskPlanGUI.ps1** — диалоговое окно управления (вкладки: Список задач, Управление, Тесты);
- **TaskPlan-Tracker.ps1** — ядро логики (работает с `ПЛАН_РЕАЛИЗАЦИИ.txt` и `plan_state.json`);
- **TaskPlan-Manager.ps1** — диспетчер множества задач (`task_index.json` , статусы активная/приостановленная);
- **TaskPlan-Tests.ps1** — 15 автотестов.

3. Работающие правила

3.1. Правило TASK

Ключевое правило запуска обработки задачи:

- определение корневого пути по типу задачи;
- поиск папки с префиксом `<имя_задачи>_*` (не создавать новую, если существует);
- получение описания из Jira через `jira_get_issue`.

3.2. Правило остановки/продолжения работы

- **Остановка** — по команде пользователя или при неясности дальнейших шагов; фиксация через MCP (точка возобновления + команда продолжения) и якорь в `rules/pending-tasks.md`.

- **Возобновление** — только по явной команде, через Knowledge First: `ais-catalog_search_knowledge` → чтение `Describe\` → продолжение с раздела «СЛЕДУЮЩИЙ ШАГ».

3.3. Knowledge First (MCP-поиск)

Перед любым не-простым запросом выполняется поиск в базах знаний в строгом приоритете (правило Token Economy):

1. **wiki-mcp** — локальная вики;
2. **ais-catalog** — база знаний AIS;
3. **sybase-docs** — документация ASE/PB;
4. **rules-mcp** — правила проекта;
5. **atlassian** — Confluence/Jira (последний резерв).

3.4. Правило документации (тройной формат)

При создании/обновлении `.md`-документации одновременно формируются `.html` и `.pdf` через `scripts\Convert-Docs.ps1`. Требования: белый/светлый фон, тёмный текст, шрифт с кириллицей, чёткие таблицы, полные пути.

3.5. Правило кодировок

Тип файла	Кодировка	ВОМ
<code>.ps1</code>	UTF-8	с BOM

<code>.json</code> / <code>.jsonc</code>	UTF-8	без BOM
<code>.sql</code> / <code>.bat</code>	Windows-1251	нет
<code>.sr*</code> (экспорт PB)	Windows-1251	нет
<code>.md</code>	UTF-8	без BOM

3.6. Правило ЗАПОМНИ (куда писать правила)

- Статическое правило для всех сессий → файл (AGENTS.md, .mdc);
- Опыт конкретной ситуации → MCP-знание (opencode-mcp / ais-catalog).

3.7. Прочие обязательные правила

- **Безопасность секретов** — логины/пароли/токены только в `config/.local_secrets.json` (в `.gitignore`), никогда не попадают в Git/GitHub;
- **Безопасность данных SQL** — при диагностике выводить только ID, коды, статусы; запрещены имена клиентов, номера полисов/договоров, суммы, адреса, телефоны, email;
- **Preflight** — проверка BOM/кодировки/идиом PB/SQL перед записью файлов;
- **Антивирус** — запрещённые паттерны PS (Bypass, IEX, base64-обёртки, скрытые окна), обязательный BOM для .ps1.

4. Работающие сервисы

Сервис	Скрипт/модуль	Назначение
TaskPlan	Show-TaskPlanGUI.ps1	ДО управления этапами объёмных задач
Tracker	TaskPlan-Tracker.ps1	Логика планов, plan_state.json
Manager	TaskPlan-Manager.ps1	Диспетчер многих задач, task_index.json
OUTPL	Out-TaskPlan.ps1	Анализ и автоисправление ошибок TaskPlan
СБОРКА	Sborka-Golden.ps1 , Show-Sborka-GUI.ps1	Сборка golden.exe + PBD; режим GALAXY (суффикс _CHAGA)
ПРОДОБР	Show-Prod0br-GUI.ps1	Продолжение обработки по утверждённому плану
ОБР/OUT	Out-LastOutput.ps1	Анализ свежего вывода (МОРДА / МОРДА2)
МОРДА	Главное окно GUI	Диалоговое окно обработки (стандарты STANDART1/STANDART2)
СОХР	Save-Snapshot.ps1	Архив проекта (только по команде)
Документация	Convert-Docs.ps1	md → html + pdf
ВОМ/антивирус	Add-Bom.ps1 , Preflight-Antivirus.ps1	Контроль ВОМ, защита от удаления антивирусом
Сохранение промптов	Save-UserPrompt.ps1	Автоархив промптов при любом ответе
Сохранение анализов	Save-Analysis.ps1	Автосохранение дорогих анализов

5. Стратегия работы с LLM

Объёмные задачи решаются недорогими/бесплатными LLM, дорогие мощные модели запускаются точечно:

- 1. **Классификация** задачи — короткая или объёмная (признаки: имя задачи, > 3 этапов, > 3 объектов);
- 2. **Составление общего плана** — для объёмных задач создаётся `ПЛАН_РЕАЛИЗАЦИИ.txt` ;
- 3. **Поэтапное выполнение с фиксацией** — после каждого этапа обновляются план, описание, запись MCP;
- 4. **Переход на другую LLM** при исчерпании ресурса — фиксация текущего этапа, возобновление только по команде;
- 5. **Откладывание непосильных подзадач** — этап помечается `[!]` , выносится в ОЧЕРЕДЬ ДОРОГИХ ПОДЗАДАЧ;
- 6. **Подготовка брифа** — перед запуском дорогой LLM собирается пакет: точный контекст, карта зависимостей, сужение задачи, критерий приёмки.

6. Ключевые команды

Команда	Действие
TASK <имя>	Начать задачу (папка, выгрузка оригиналов, план)
PRODOBR	Продолжить обработку по утверждённому плану
TaskPl	Открыть ДО управления TaskPlan
OUTPL	Анализ ошибок TaskPlan
SBORKA [GALAXY/DEBUG]	Сборка golden.exe + PBD
OTCHET	Открыть отчёт по задаче

OUT / OBR	Анализ свежего вывода
DLLM	Список/статус моделей
GITHUB	Обновить git + push в GitHub
RR	Принудительно русский язык

7. Выводы

- Процесс полностью построен вокруг **Jira + файловой структуры папок** без тяжёлой проектной документации.
- **SDD заменён** на `ПЛАН_РЕАЛИЗАЦИИ.txt`, `Describe\` и MCP-записи — лёгкие, поддерживаемые любой LLM-сессией артефакты.
- **Правила** охватывают: запуск задач, остановку/возобновление, поиск знаний, кодировки, безопасность секретов и данных, документирование.
- **Сервисы** автоматизируют: управление этапами, сборку, анализ вывода, автоподмену моделей, документацию.
- Ключевой принцип — **продолжаемость**: любая новая LLM-сессия может продолжить задачу с «СЛЕДУЮЩЕГО ШАГА» без потери контекста.